Linux System Administration (E1ITA307)

Linux Infrastructure Deployment Project Report

On

LINUX STORAGE MONITORING SYSTEM

Submitted by

KOUSHIK K T (2411021060147)

NEHAA MAHESH (2411021060103)

SHAURYAMAN SINGH (2411021060077)

In partial fulfillment of the award of the degree of

BACHELOR OF TECHNOLOGY

In

Information Technology

Under the Supervision of

Prof. Kusuma J

ALLIANCE SCHOOL OF ADVANCED COMPUTING

BENGALURU

October 2026



DEPARTMENT OF INFORMATION TECHNOLOGY

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the Project work entitled **"Linux Storage Monitoring System"** submitted by Koushik K T [2411021060147], Nehaa Mahesh [2411021060103] and Shauryaman Singh [2411021060077] in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology-IT of Alliance University, is a bonafide work during the academic year 2026-2027.

Prof. Kusuma J	Prof. Ananthanagu U	Dr. Shekhar R
Course Handling Faculty	Course Coordinator	Professor & HoD

Table of Contents

ABSTRACT

CHAPTER 1 — INTRODUCTION

1.1 Aim

1.2 Background

1.3 Problem Statement

1.4 Objectives

1.5 Scope of the Project

CHAPTER 2 — METHODOLOGY

2.1 Tools and Technologies Used

2.2 Hardware and Software Requirements

2.3 System Requirements and Design Approach

2.4 Development Process

CHAPTER 3 — LINUX STORAGE MONITORING SYSTEM - DESIGN AND IMPLEMENTATION

3.1 System Architecture

3.2 Linux Environment Setup

3.3 Project Directory Structure

3.4 Web Application and Backend Implementation

3.5 Storage and Filesystem Monitoring

3.6 Inode Monitoring and Threshold Configuration

3.7 Alert Simulation and Log Management

3.8 Linux Command Integration

3.9 Storage Report Generation

3.10 Security and Access Control

CHAPTER 4 — RESULTS AND TESTING

4.1 Implementation Results

4.2 Dashboard and Storage Monitoring Results

4.3 Testing and Validation

4.4 Limitations of the System

CHAPTER 5 — CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

5.2 Future Enhancements

REFERENCES

APPENDIX A — PROJECT DIRECTORY STRUCTURE

APPENDIX B — REPRESENTATIVE LINUX COMMANDS

APPENDIX C — SCREENSHOT PLACEHOLDERS

ABSTRACT

Storage management is an important part of Linux system administration because insufficient disk space and high inode utilization can affect system performance and prevent applications from creating new files. Manually checking storage information through terminal commands can be time-consuming and may make it difficult to identify potential storage issues quickly.

The **Linux Storage Monitoring System** is a web-based application developed to simplify storage monitoring in a Linux environment. The application uses Python and Flask to provide a browser-based interface connected to Linux system utilities. It collects filesystem information, displays disk usage and available storage, monitors inode utilization, and identifies warning or critical conditions using configurable thresholds.

The dashboard provides a Linux command console with predefined storage-related commands, including `df -hT`, `df -i`, `lsblk`, and `findmnt`. Additional features include configurable warning and critical thresholds, alert simulation, activity logging, and downloadable storage reports. The alert simulation allows users to test different storage utilization levels without modifying actual disk contents.

The system is designed to run locally within a Kali Linux environment and provides a simple interface for viewing storage information and performing basic monitoring tasks. It combines Linux command-line utilities with a web interface to make storage information easier to understand and access.

This project demonstrates the practical application of Linux system administration, filesystem monitoring, backend development, and threshold-based alerting. It can be extended in the future with scheduled monitoring, historical usage graphs, automated notifications, and additional storage management capabilities.

CHAPTER 1 — INTRODUCTION

1.1 Aim

The main aim of this project is to develop a web-based Linux Storage Monitoring System that provides clear and accessible information about filesystem utilization, available disk space, inode usage, and storage-related alerts.

The application connects a browser-based dashboard to a Python Flask backend, which executes predefined Linux commands to collect system information. It also provides configurable warning and critical thresholds, alert simulation, log viewing, and storage report generation. The project aims to simplify basic storage monitoring and demonstrate the use of Linux system administration tools through a web interface.

1.2 Background

Linux operating systems are widely used in servers, development environments, virtual machines, and cloud infrastructure. Effective storage monitoring is necessary to ensure that filesystems have sufficient free space and that system resources remain available for applications.

Linux provides utilities such as `df`, `df -i`, `lsblk`, and `findmnt` to inspect filesystem capacity, inode utilization, block devices, and mounted filesystems. However, users must interpret their output manually when working entirely through the terminal.

The proposed system combines these Linux utilities with a web-based dashboard. This allows users to view storage information, inspect command output, configure usage thresholds, and generate reports from a single interface.

1.3 Problem Statement

Linux administrators need to monitor disk space and inode availability to prevent storage-related problems. When storage utilization becomes high, applications may fail to create files, logs may stop updating, and system operations may be affected.

Traditional command-line monitoring requires users to execute multiple commands and interpret their output separately. This can make regular monitoring less convenient, particularly for beginners.

Therefore, this project addresses the need for a simple web-based interface that collects storage information from Linux, presents it in a readable format, identifies threshold violations, and supports basic reporting and log inspection.

1.4 Objectives

The main objectives of the project are:

1. To develop a web-based dashboard for Linux storage monitoring.
2. To display filesystem capacity, used space, available space, and utilization percentage.
3. To monitor inode usage and identify high utilization.
4. To integrate predefined Linux storage commands with the web interface.
5. To configure warning and critical usage thresholds.
6. To simulate storage alerts without modifying actual disk contents.
7. To record application activity in log files.
8. To generate downloadable storage reports.
9. To demonstrate the practical application of Linux commands and Python-based web development.

1.5 Scope of the Project

The project focuses on monitoring existing filesystems within the Linux environment in which the application runs. It provides a dashboard for disk and inode information, predefined command execution, threshold configuration, alert simulation, log viewing, and report generation.

The application is intended for local educational and demonstration purposes. It does not currently perform automatic disk cleanup, partition creation, filesystem resizing, or automated scheduled monitoring. These functions may be considered future enhancement.

CHAPTER 2 — SYSTEM REQUIREMENTS AND METHODOLOGY

2.1 Tools and Technologies Used

Kali Linux: The operating system used to run the application and provide access to Linux storage utilities.

Python 3: The programming language used to implement the application backend.

Flask: A lightweight Python web framework used to serve the dashboard and provide API endpoints.

HTML and CSS: Used to structure and style the browser-based interface.

JavaScript: Used to request storage data, update dashboard information, execute predefined commands through the backend, and handle user interactions.

Linux Utilities: The `df`, `lsblk`, and `findmnt` commands are used to inspect filesystems, disk usage, block devices, and mount points.

JSON: Used to store configurable warning and critical thresholds.

Log and Report Files: Used to retain application activity and generated storage reports.

Oracle VirtualBox: Used to run Kali Linux inside the Windows host computer.

2.2 Hardware and Software Requirements

Hardware requirements

- A laptop or desktop computer.
- A compatible processor capable of running Oracle VirtualBox.
- Sufficient RAM to run Windows, the virtual machine, and the web application.
- Available storage for the virtual machine and project files.

Software requirements

- Windows as the host operating system.
- Oracle VirtualBox.
- Kali Linux as the guest operating system.
- Python 3 and Python virtual environment support.
- Flask.
- A web browser such as Firefox.
- A text editor or terminal for creating and maintaining project files.

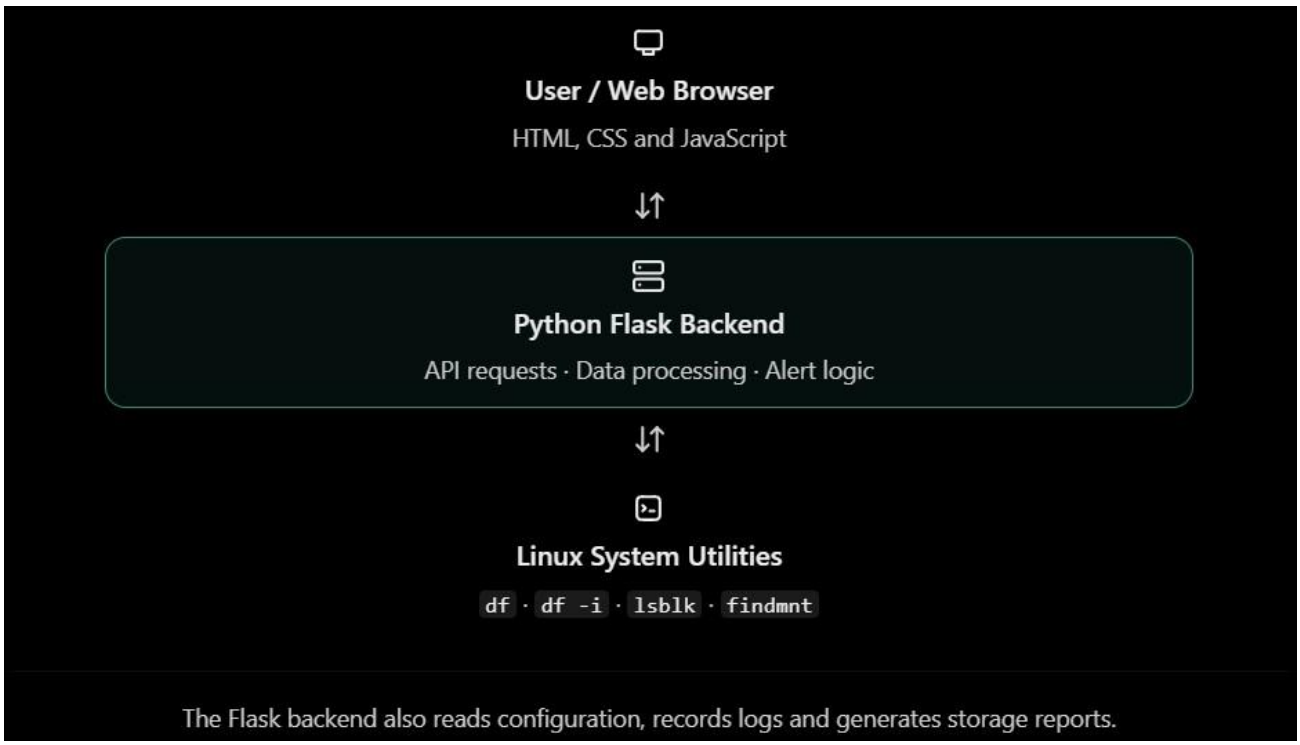
An internet connection is useful for installing dependencies and loading Google Fonts, but the local monitoring functions can operate without external web access once the required software is installed.

2.3 System Requirements and Design Approach

The application follows a simple client-server architecture. The frontend runs in a browser and communicates with the Flask backend through HTTP requests. The backend executes a predefined set of Linux commands, processes the output, and returns the information in JSON format.

The frontend displays the returned data as storage cards, tables, alerts, and terminal output. Configuration settings are stored in a JSON file, while application activity is recorded in a log file. Storage reports are generated as downloadable text files.

The command interface uses an allowlist of predefined commands instead of accepting arbitrary shell input. This reduces the risk associated with exposing terminal execution through a web application.



2.4 Development Process

The development process was divided into the following stages:

1. **Requirement analysis:** Identifying the need for filesystem monitoring, inode monitoring, alerts, logs, and reports.
2. **Environment setup:** Preparing Kali Linux and installing Python and Flask.
3. **Backend development:** Creating API endpoints to collect Linux storage information and manage configuration.
4. **Frontend development:** Designing the dashboard with storage cards, tables, command buttons, threshold settings, and report controls.
5. **Integration:** Connecting the browser interface to the Flask backend.
6. **Testing:** Checking storage outputs, threshold settings, simulation results, logging, and report downloads.
7. **Documentation:** Recording the architecture, implementation process, screenshots, and testing results.

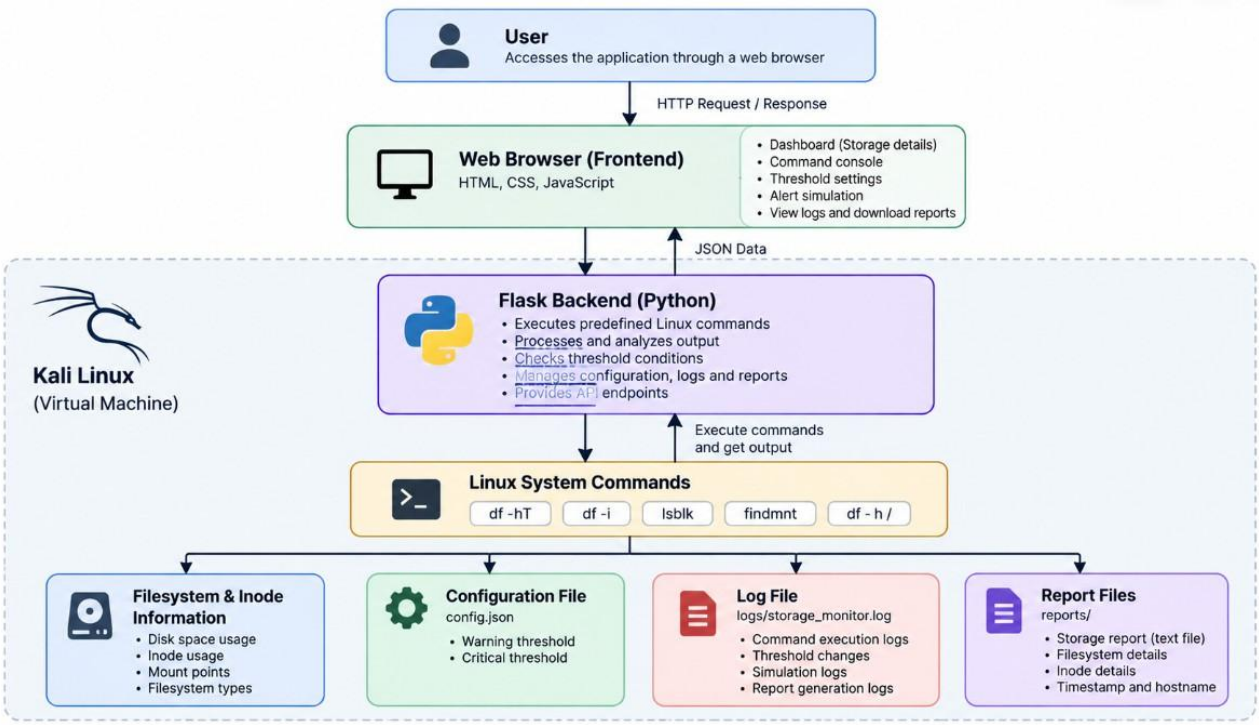
CHAPTER 3 — DESIGN AND IMPLEMENTATION

3.1 System Architecture

The Linux Storage Monitoring System consists of a browser-based frontend, a Python Flask backend, Linux system utilities, configuration storage, application logs, and report files.

The user interacts with the dashboard through a web browser. The frontend sends requests to the Flask backend, which executes approved Linux commands and processes their output. The backend returns filesystem and inode information to the dashboard. The application also manages threshold settings, records selected activities in log files, and creates downloadable reports.

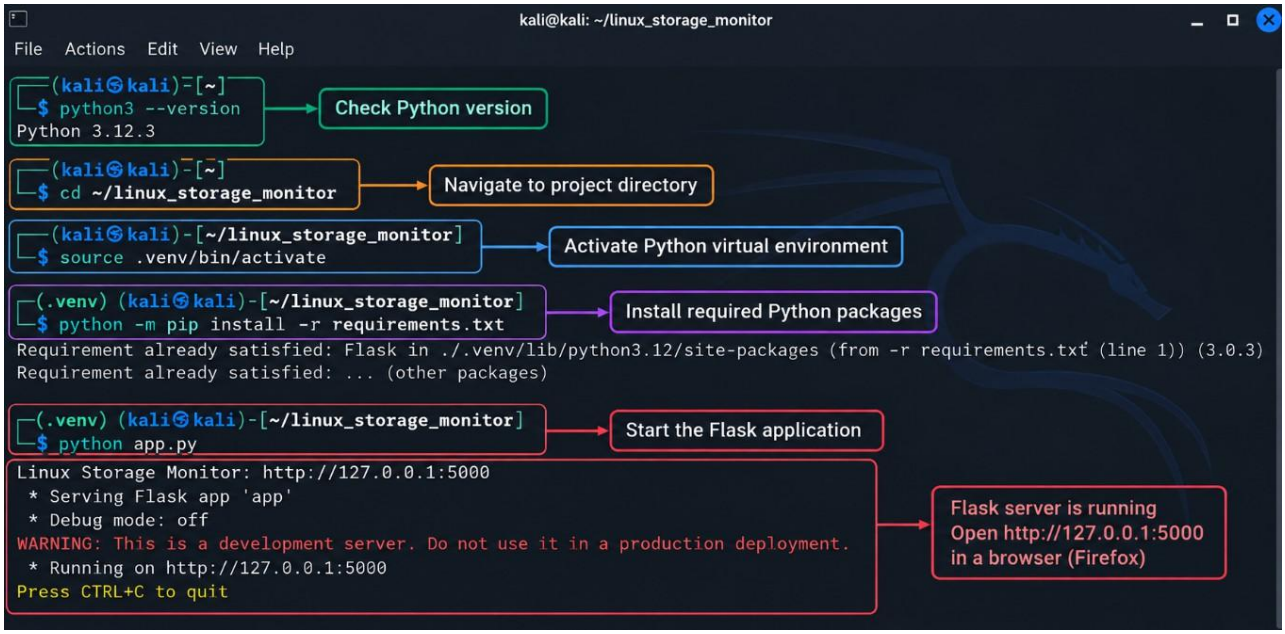
The system operates locally within Kali Linux. Oracle VirtualBox provides the virtualized environment in which the application runs.



3.2 Linux Environment Setup

The application was developed and executed inside a Kali Linux virtual machine running on Oracle VirtualBox. Python 3 was used to execute the backend, while Flask provided the web server and API functionality.

The project directory was created inside the user's home directory. A Python virtual environment was used to isolate the application's dependencies. After Flask was installed, the application was started using the Python interpreter, and the dashboard was accessed through the local address `http://127.0.0.1:5000`.

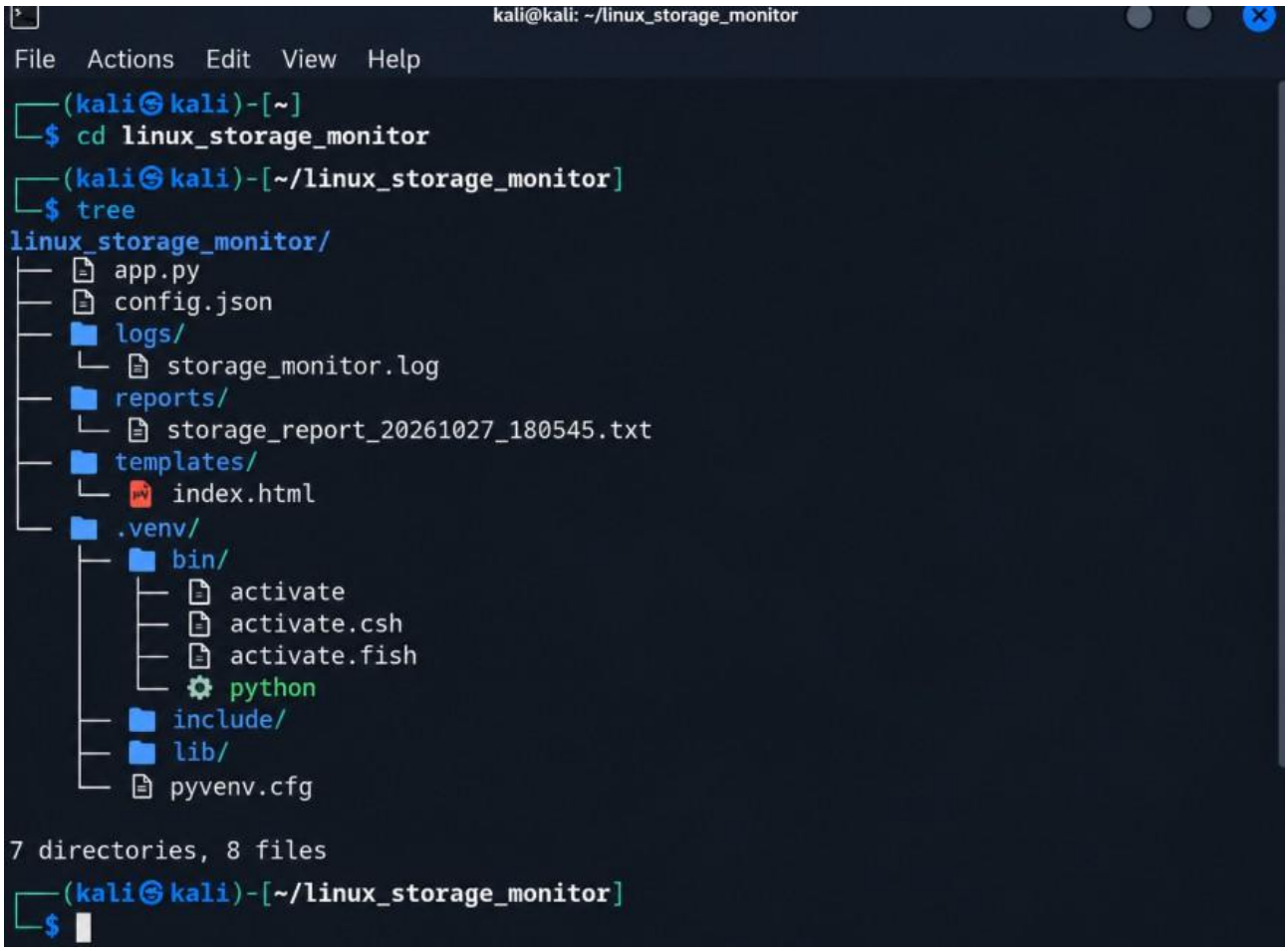


3.3 Project Directory Structure

The project uses a structured directory layout to separate the backend application, webpage templates, configuration, logs, and generated reports. This organization makes the project easier to

understand, maintain, and troubleshoot.

The main Python file contains the Flask routes and the dashboard template. The configuration file stores threshold values, the logs directory contains application activity, and the reports directory stores generated storage reports.



3.4 Web Application and Backend Implementation

The frontend provides a dashboard for viewing filesystem information and accessing monitoring functions. It is built using HTML, CSS, and JavaScript. The interface includes storage summary cards, filesystem tables, inode information, alert indicators, threshold settings, a command console, log output, and report download functionality.

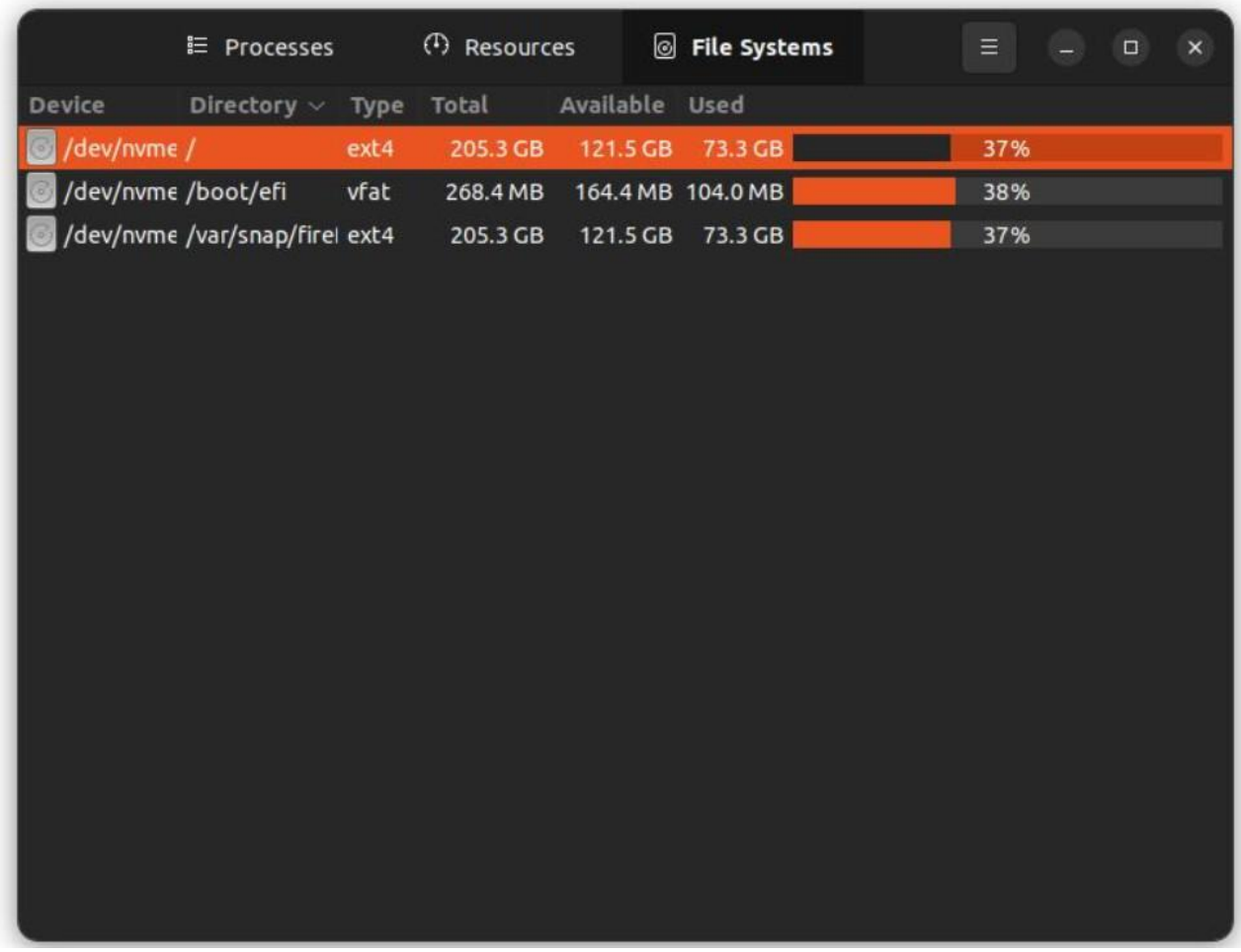
The backend is implemented using Flask. It provides routes for rendering the dashboard, returning storage information, executing predefined commands, updating thresholds, simulating alerts, retrieving logs, and generating reports. JavaScript requests data from these routes and updates the

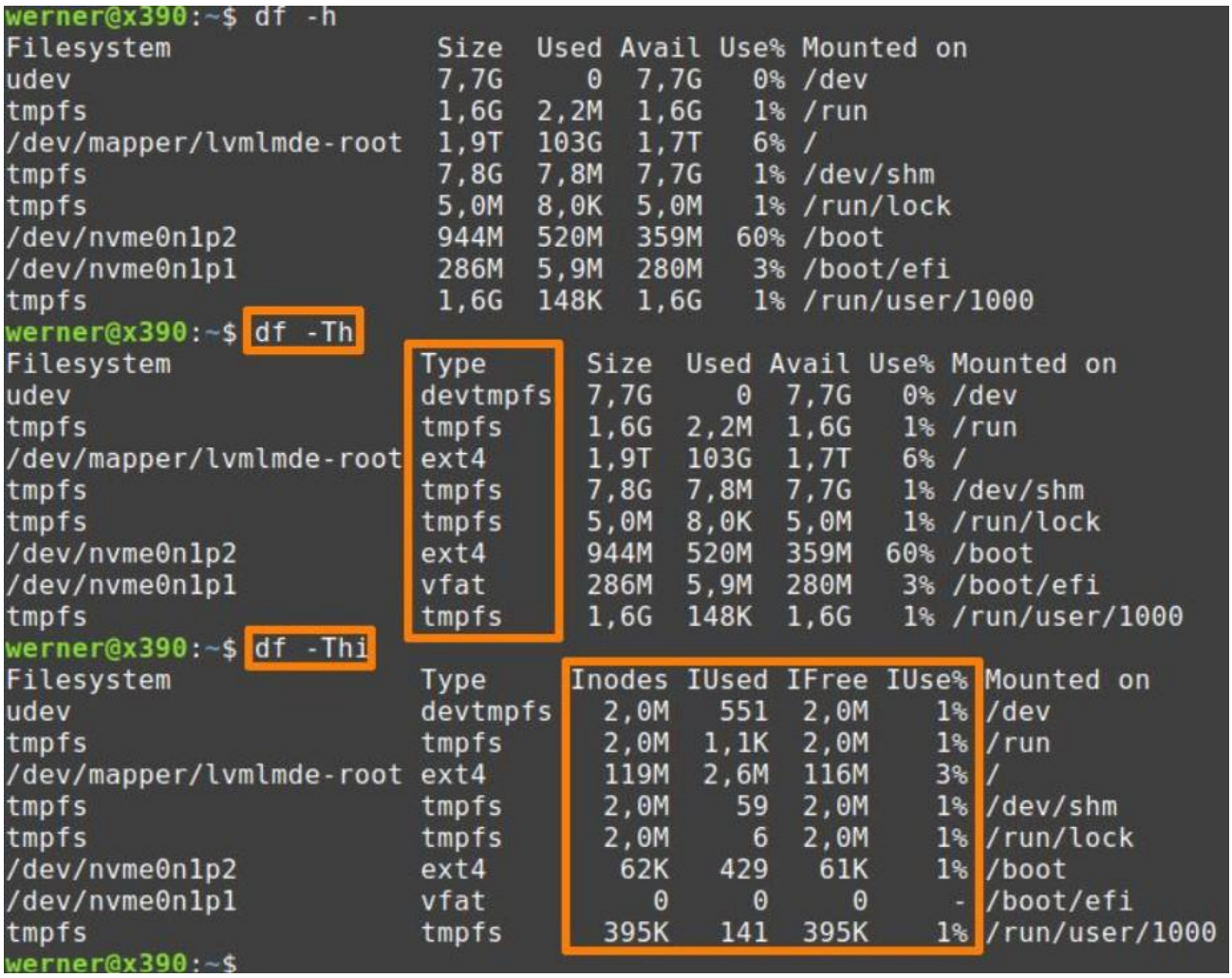
page without requiring a full page reload for each monitoring action.

3.5 Storage and File System Monitoring

Filesystem monitoring is the core function of the application. The backend uses Linux storage utilities to collect information about filesystem types, capacity, used space, available space, mount points, and utilization percentages.

The `df` command provides filesystem usage information, while `lsblk` displays block devices and their associated properties. The `findmnt` command displays mounted filesystems. The collected information is processed and returned to the frontend, where users can inspect individual filesystems and identify those with relatively high utilization.



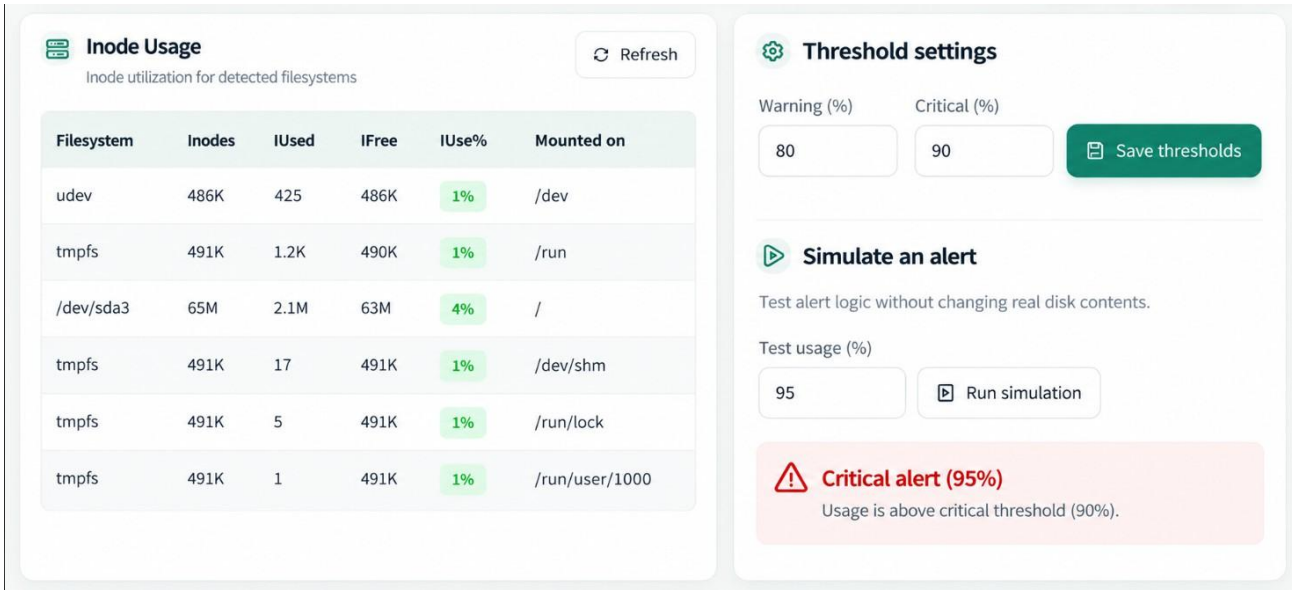


3.6 Inode Monitoring and Threshold Configuration

Inodes store filesystem metadata associated with files and directories. A filesystem may encounter inode exhaustion even when some disk space remains available.

The application uses the `df -i` command to retrieve inode utilization. It compares disk and inode usage percentages with configurable warning and critical thresholds. The default configuration uses a warning threshold of 80% and a critical threshold of 90%.

These values can be changed through the dashboard and stored in `config.json`. The application categorizes utilization as normal, warning, or critical based on the configured thresholds.



3.7 Alert Simulation and Log Management

The application provides an alert simulation function to demonstrate how the system responds to different storage utilization levels. Users enter a test percentage, and the backend evaluates it against the configured thresholds.

The simulation does not fill the disk or modify actual filesystem contents. Instead, it demonstrates the alert classification logic and records the simulation event in the application log.

The log interface allows users to inspect recorded application activities, such as command execution, threshold changes, report generation, and simulated alerts. This provides a basic record of the application's operations.

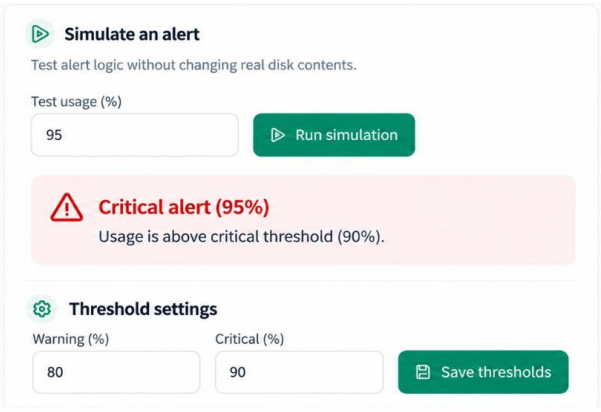


Figure 3.10: Alert simulation showing a 95% critical result.

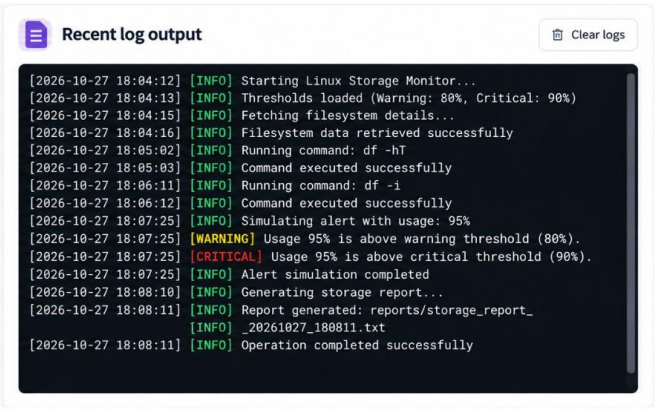


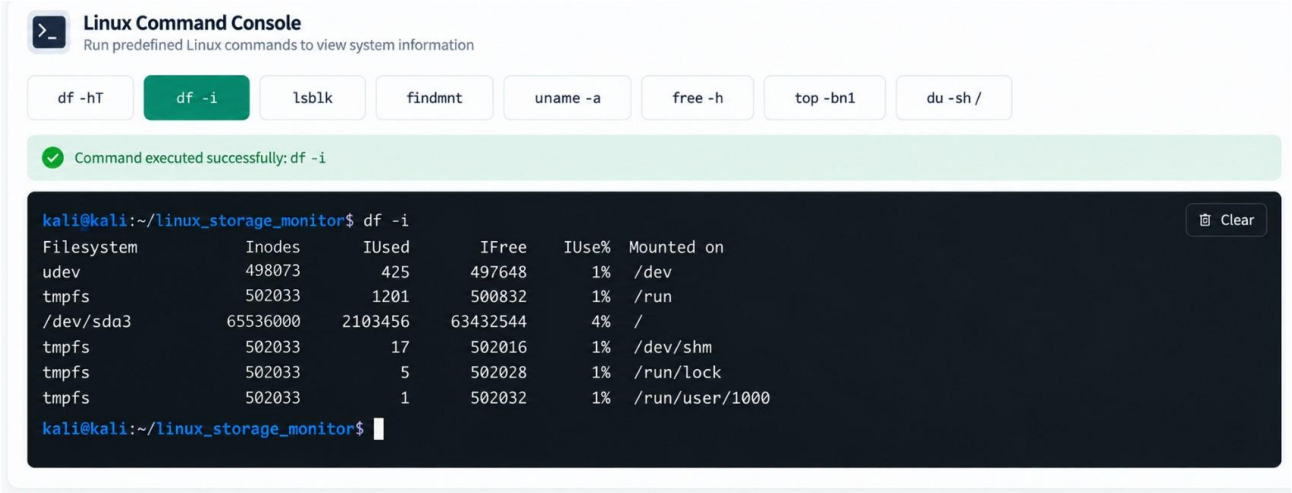
Figure 3.11: Recent log output after running a simulation or command.

3.8 Linux Command Integration

The application integrates Linux command-line utilities with the browser-based interface. Users can select predefined commands from the command console and view the output returned by the backend.

The supported commands include `df -hT` for human-readable filesystem usage and filesystem types, `df -i` for inode utilization, `lsblk` for block device information, `findmnt` for mounted filesystem details, and `df -h /` for root filesystem usage.

The backend uses an allowlist to restrict command execution to these predefined operations. This design demonstrates how a web application can provide controlled access to Linux system information without exposing an unrestricted terminal.



3.9 Storage Report Generation

The application includes a report generation function that collects filesystem and inode information and writes it to a text report. The report includes the host identification, generation timestamp, configured thresholds, filesystem information, and inode utilization details.

The generated report can be downloaded through the dashboard. This makes it possible to preserve a snapshot of the system's storage status for review or inclusion in project documentation.

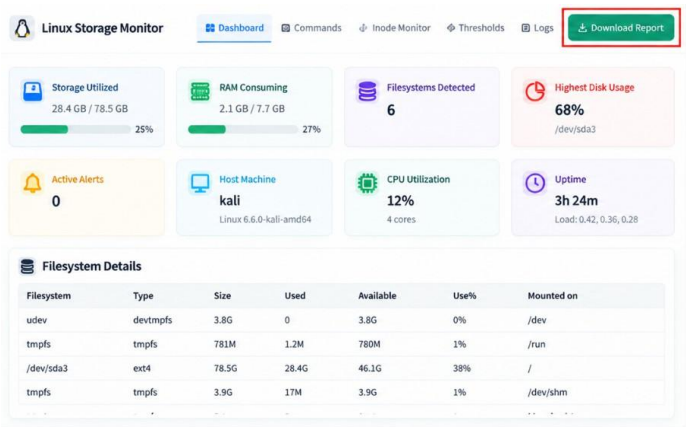


Figure 3.13: Dashboard with the Download Report control.

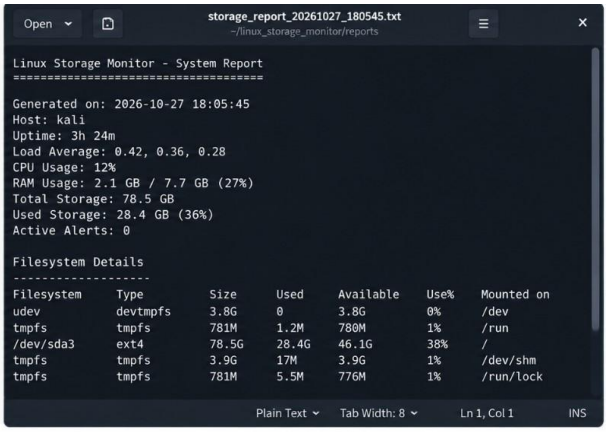
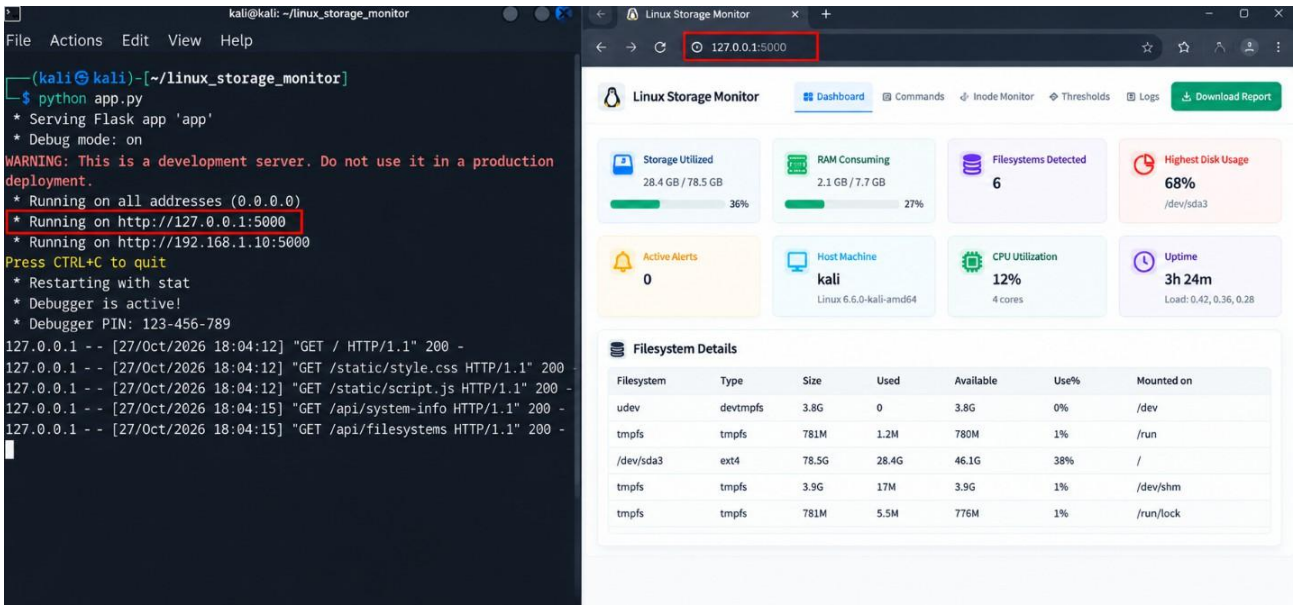


Figure 3.14: Generated text report opened in a text editor.

3.10 Security and Access Control

The application is designed for local educational use. The Flask server binds to the loopback address `127.0.0.1`, making the dashboard accessible from the same Linux environment by default. The command console executes only predefined storage-related commands rather than arbitrary shell commands.

The application does not require privileged commands for its normal monitoring functions. This reduces the risk of unintended system modifications. However, the application is not a production-ready administrative platform and does not currently include user authentication, role-based access control, or a complete security audit



CHAPTER 4 — RESULTS AND TESTING

4.1 Implementation Results

The implemented application combines Linux storage utilities with a web-based dashboard. It provides filesystem and inode monitoring, configurable thresholds, alert simulation, predefined command execution, log viewing, and downloadable storage reports.

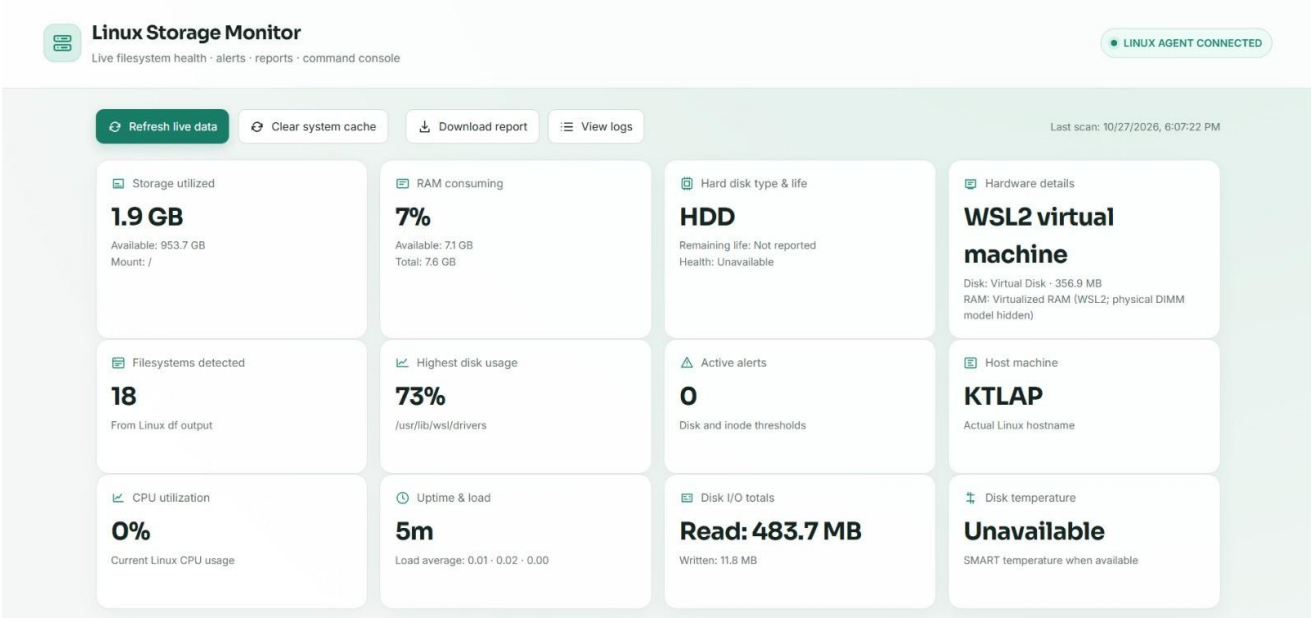
The dashboard presents storage information in a structured format, allowing users to inspect filesystem utilization and access related monitoring functions from one interface. The alert simulation provides a safe way to demonstrate threshold-based classification without altering actual disk usage.

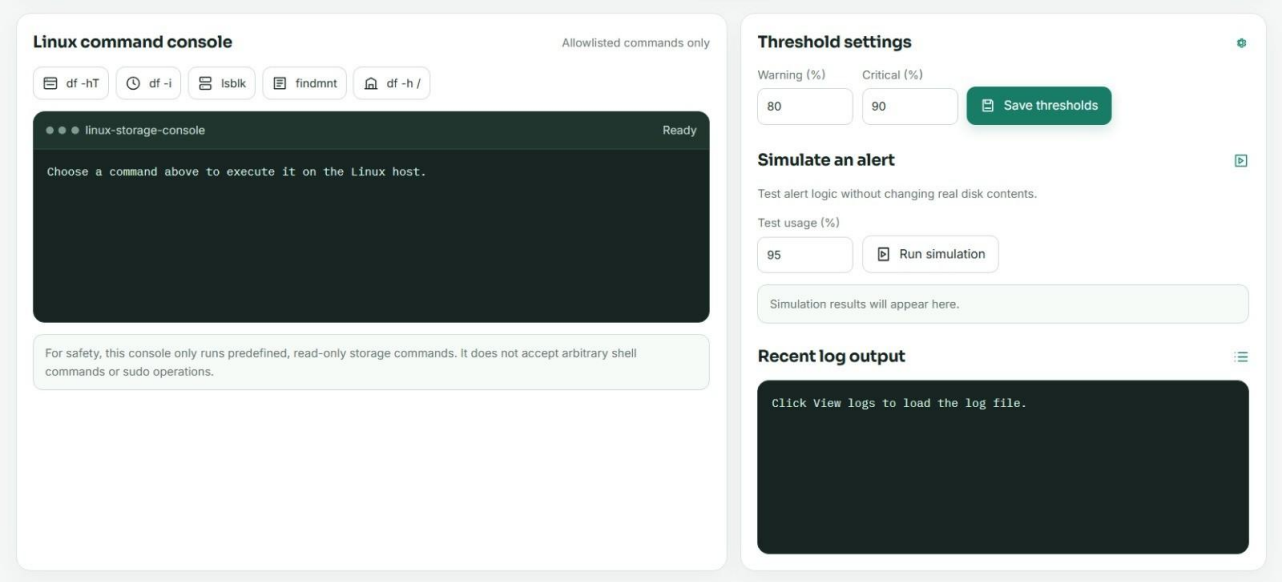
The final results should be documented using screenshots and test outputs collected from the running Kali Linux application.

4.2 Dashboard and Storage Monitoring Results

The dashboard displays the filesystem information collected from the Linux environment. Summary cards show the number of detected filesystems, the highest disk utilization, the number of active threshold alerts, and the hostname.

Detailed tables present filesystem capacity, used and available space, mount points, inode information, and utilization status. The command console provides direct access to the supported storage commands, while the report and log functions support inspection of monitoring activity.





4.3 Testing and Validation

Test Case	Expected Result	Actual Result/Status
Dashboard Launch	Dashboard opens at 127.0.0.1:5000	Dashboard Opens
FileSystem Monitoring	Filesystem Data is Displayed	Data is Displayed
Inode Monitoring	Inode Utilization is Displayed	Inode Utilization is Displayed
Command Execution	Command Output Appears	Command Output Appears
Threshold Update	Valid Threshold values are saved	Values are saved
Alert Simulation	Correct status is returned for test percentage	Done
Log Viewing	Recorded application activity is displayed	Activity is Displayed
Report Generation	Text report downloads successfully	Downloaded Successfully

Invalid Simulation I/P	Out-of-range percentage is rejected	Done

4.4 Limitations of the System

The current implementation focuses on local filesystem monitoring and basic reporting. It does not automatically delete files, resize partitions, create filesystems, or perform storage cleanup. Alert simulation demonstrates threshold logic but does not generate real disk pressure.

The application does not currently provide scheduled monitoring through Cron, long-term historical graphs, automated email notifications, or multi-user authentication. Hardware-specific details such as disk health and temperature may also be unavailable in a virtualized environment.

These limitations define the current scope of the mini project and provide opportunities for future development.

CHAPTER 5 — CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

The Linux Storage Monitoring System demonstrates how Linux command-line utilities can be integrated with a web-based interface to simplify filesystem monitoring. By combining Python, Flask, HTML, CSS, JavaScript, and standard Linux commands, the project provides a centralized dashboard for viewing disk space, inode utilization, threshold alerts, command output, logs, and storage reports.

The project applies practical concepts of Linux system administration, filesystem monitoring, backend development, API communication, configuration management, and basic security. The alert simulation function also allows threshold behavior to be demonstrated without modifying actual disk contents.

Overall, the project provides a foundation for a more comprehensive storage administration platform. Testing the implemented features and documenting their actual outputs demonstrates the practical value of connecting Linux system utilities with a simple web application.

5.2 Future Enhancements

The system can be improved through the following enhancements:

1. **Automated monitoring:** Integrate Cron or a background service to perform scheduled storage checks.
2. **Historical usage graphs:** Store measurements over time and display disk utilization trends.
3. **Real-time notifications:** Send email or other notifications when usage crosses critical thresholds.
4. **Storage analysis:** Identify directories consuming the most space using controlled `du` operations.
5. **Backup integration:** Add backup creation and verification for selected project files.
6. **Authentication:** Introduce user login and role-based permissions for administrative functions.

7. **Multi-system monitoring:** Extend the architecture to monitor multiple Linux machines securely.
8. **Improved reporting:** Add CSV exports, graphical summaries, and comparisons between monitoring periods.

REFERENCES

1. The Linux man-pages project, "df(1) — report file system disk space usage." https://man7.org/linux/man-pages/man1/df.1.html
2. The Linux man-pages project, "lsblk(8) — list block devices." https://man7.org/linux/man-pages/man8/lsblk.8.html
3. The Linux man-pages project, "findmnt(8) — find a filesystem." https://man7.org/linux/man-pages/man8/findmnt.8.html
4. Pallets Projects, "Flask Documentation." https://flask.palletsprojects.com/
5. Python Software Foundation, "Python 3 Documentation." https://docs.python.org/3/
6. Kali Linux, "Kali Linux Documentation." https://www.kali.org/docs/
7. Oracle, "Oracle VirtualBox Documentation." https://www.virtualbox.org/wiki/Documentation

APPENDIX A — PROJECT DIRECTORY STRUCTURE

The project root is:

`~/linux-storage-sandbox/`

Important paths include:

```
linux-storage-sandbox/
├── sandbox.sh
├── config/
├── data/
├── registry/
│   └── sandboxes.db
├── backups/
├── logs/
├── modules/
├── scripts/
├── tests/
│   └── run_tests.sh
└── docs/
```

This is a simplified overview of the project structure.

APPENDIX B — REPRESENTATIVE LINUX COMMANDS

Inspect the project directory:

```
cd ~/linux-storage-sandbox
ls -lah
```

Inspect sandbox directories:

```
ls -lah data/
```

Inspect registry records:

```
cat registry/sandboxes.db
```

Measure sandbox directory usage:

```
du -sh data/
du -sb data/
```

Inspect filesystem capacity:

```
df -h ~/linux-storage-sandbox
```

Run the CLI test suite:

```
./tests/run_tests.sh
```

These commands are provided as examples of Linux inspection and verification.

APPENDIX C — WEB IMPLEMENTATION AND SYSTEM DEMONSTRATION

C.1 Introduction to the Web Interface

The web interface extends the command-line implementation by providing a graphical environment for viewing sandbox information and monitoring system activity. It consists of a React/TypeScript frontend that communicates with a FastAPI backend, which in turn interacts with the existing Linux project implementation.

The interface is intended to make system information easier to view and administrative operations easier to access. The web interface complements the Linux CLI rather than replacing the underlying Bash-based implementation.

C.2 Web Dashboard

The dashboard provides an overview of the sandbox environment and presents relevant system information in a consolidated interface, as shown in Figure C.1.

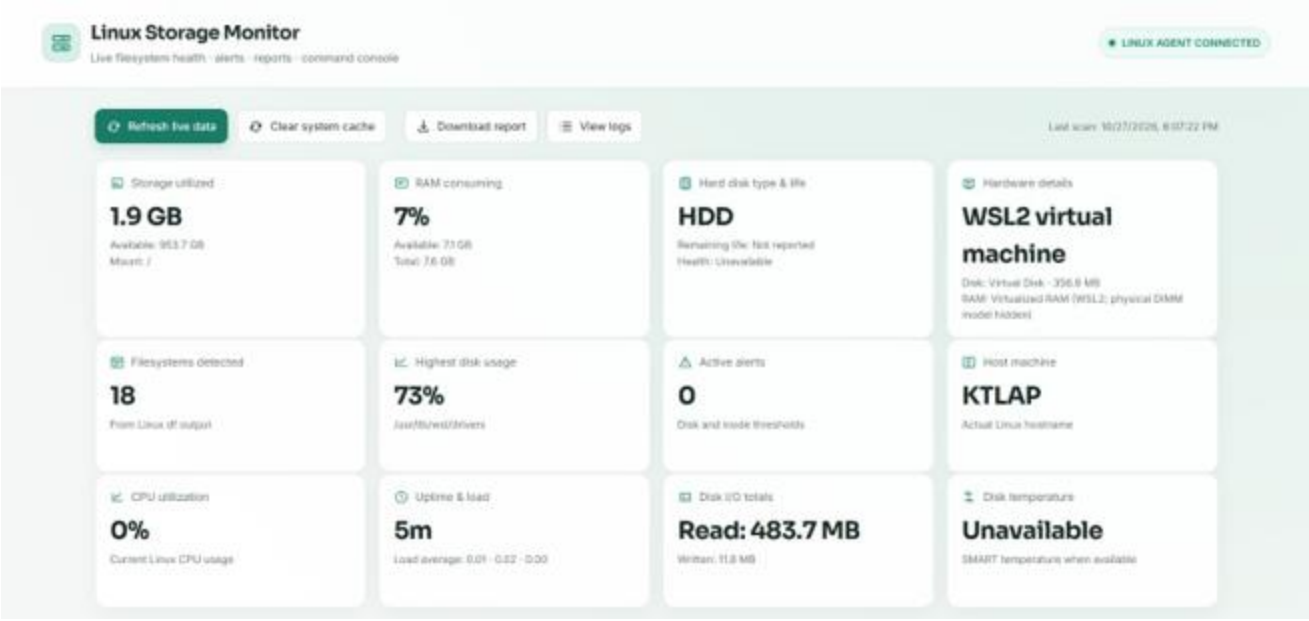
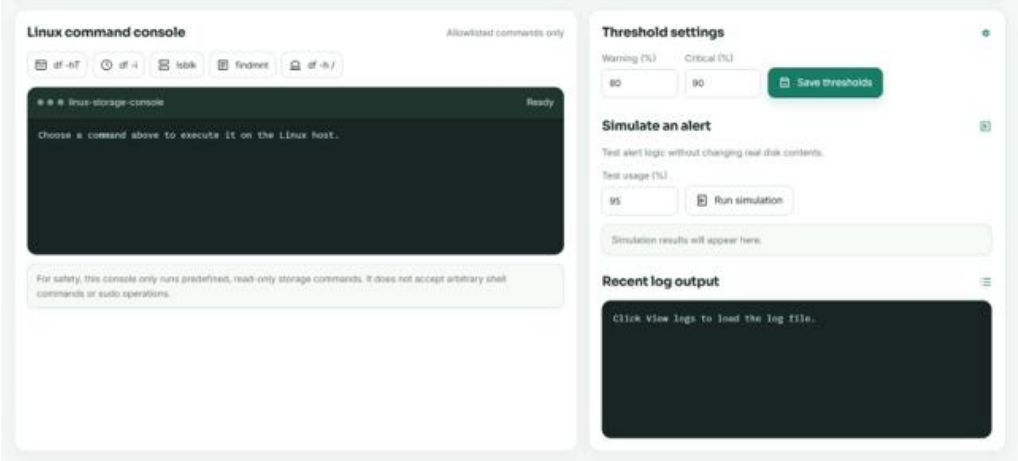


Figure C.2: Web Dashboard Overview

C.3 Sandbox Management Interface

The sandbox management interface presents the registered sandbox environments and supports the management actions implemented in the project, as shown in Figure C.2.

The web interface communicates with the backend to retrieve information and perform supported operations against the existing Linux implementation.



v

Figure C.3: Sandbox Management Interface

C.4 Storage Monitoring and Visualization

The storage monitoring page presents sandbox usage information and filesystem-related measurements, as shown in Figure C.3.

The configured sandbox storage targets are monitoring benchmarks and are not kernel-enforced disk quotas

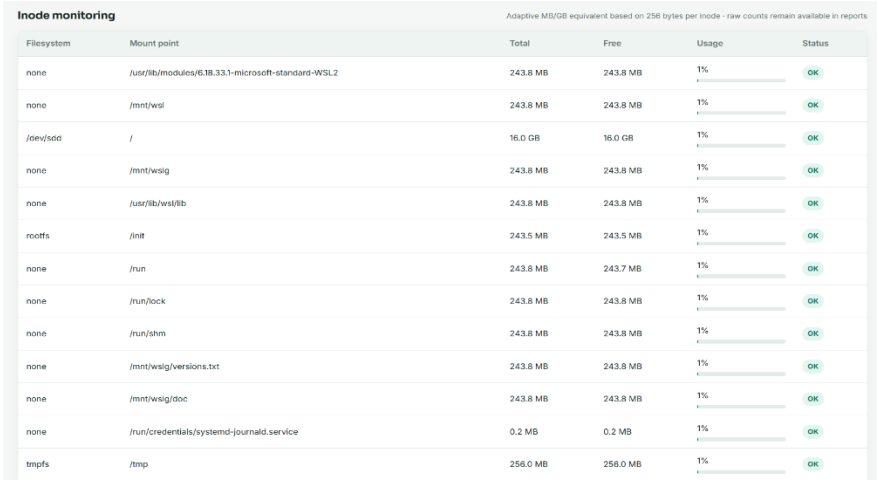


Figure C.4: Storage Monitoring and Visualization

C.5 Audit Logs and System Activity

The audit log interface presents recorded administrative activities to help review operations and troubleshoot issues, as shown in Figure C 4.

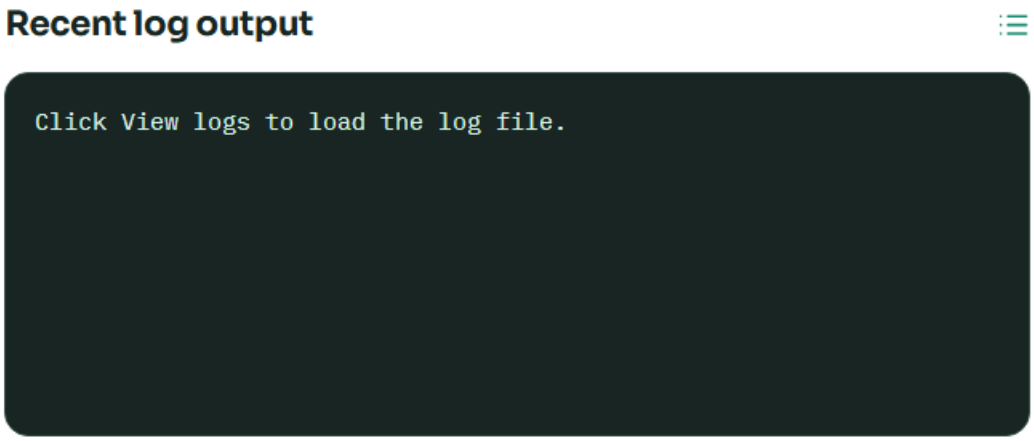


Figure C.5: Audit Log Monitoring

C.6 System Verification and Test Results

The project is verified through automated tests and backend health checks. The CLI regression suite documented in Chapter 4 passed 11 out of 11 tests. That result is distinct from any frontend or backend test results, which are reported only as supported by Figure C.7. Historical results are not presented as newly executed tests.

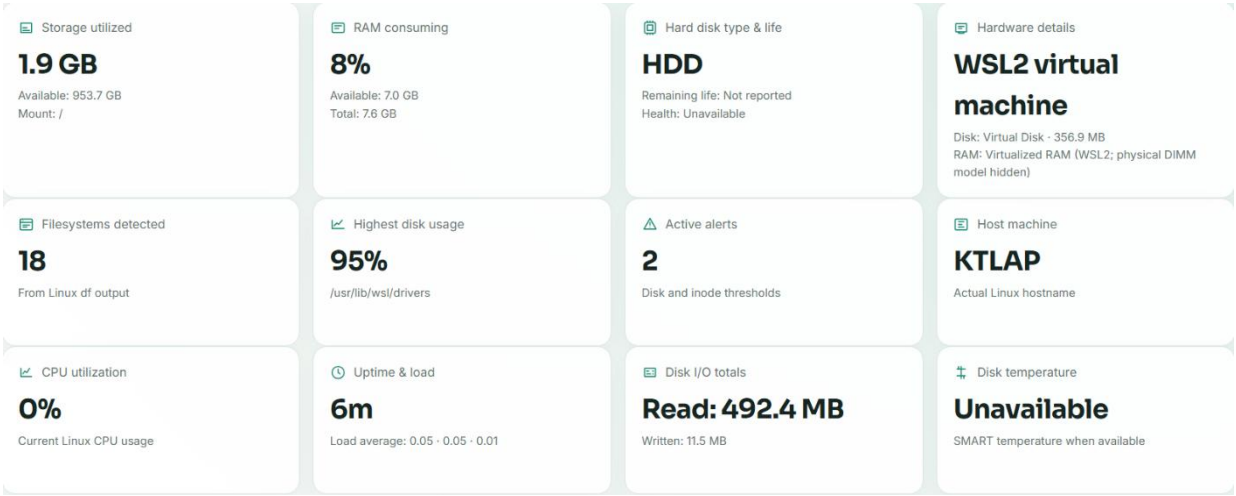


Figure C.6: System Verification and Test Results

C.7 Summary of the Web Demonstration

The web implementation provides a graphical interface for accessing the Linux Storage Sandbox Infrastructure for Isolated Enterprise Project Environments' supported functionality. By connecting the React frontend to the FastAPI backend, the project combines a user-friendly interface with the existing Linux-based administration logic.

The demonstration illustrates the integration of web technologies with Linux system administration, filesystem monitoring, sandbox management, backup and restore safeguards, audit logging, and automated verification.

The web interface supplements the command-line tools and does not change the project's fundamental distinction between monitoring storage usage and enforcing storage limits.